Implementation of Local and Longform Attention Variants for Efficient BERT Training

Jonas Alexander Lütticken/jonas.luetticken@stud.uni-heidelberg.de

30. September 2025

1 Introduction

Neural language models such as BERT (Devlin et al. 2019) achieve strong performance across a wide range of NLP tasks, but rely on dense self-attention. This mechanism scales quadratically with sequence length, which makes training and inference computationally expensive in terms of runtime and memory usage. For longer sequences, such as those in large corpora (e.g., WikiText-103), the quadratic complexity quickly becomes a bottleneck.

To address this issue, researchers have proposed *sparse attention* patterns. These restrict the number of attention connections, aiming to significantly reduce computational cost while retaining the most relevant context. Different sparsity patterns make different assumptions about which context tokens are most important.

In this project, I implement and compare two sparse attention variants against a dense baseline:

- **Baseline (Dense Attention):** Standard BERT-base masked language model.
- **Local Window Attention:** Restricts attention to a fixed neighborhood around each token, reducing complexity but limiting global context.
- **Longform Attention:** Employs a sparse pattern that allows selective long-range connections while keeping computational costs lower than dense attention.

The goal is to evaluate the trade-offs between effectiveness (measured by evaluation loss and perplexity) and efficiency (measured by runtime, tokens per second, and GPU memory usage). The central research questions are:

1. How do sparse variants compare to dense attention in terms of model quality?
2. Where are the trade-offs between performance and efficiency?

I focused on Local Window and Longform attention because they represent two contrasting design choices for sparse attention: one that strictly limits attention to local neighborhoods, and another that combines local windows with sparse global connections. This contrast makes them particularly suitable for highlighting the trade-offs between model quality and efficiency in comparison to the dense baseline. By focusing on a baseline and two contrasting sparse variants, this project demonstrates how different attention patterns affect the balance between performance and efficiency. The findings provide insight into which approaches may be more suitable for training efficient long-context language models in practical NLP settings.

2 Experimental Setup

2.1 Dataset

All experiments are conducted on the WikiText-103 corpus. For computational reasons, a subset of approximately 5 million tokens was used. The dataset was accessed via the HuggingFace `datasets` library and preprocessed with the BERT tokenizer.

Split	#Examples	#Tokens
Training	9779	4.5 Mio.
Validation	466	0.5 Mio.

Table 1: Dataset statistics for the WikiText-103 subset used in this project.

2.2 Tools and Environment

The implementation is based on PyTorch 2.x and the HuggingFace Transformers library. All training was performed using an NVIDIA A100 GPU with 40GB VRAM. All runtime and memory measurements were recorded under these conditions.

2.3 Evaluation Metrics

The models are compared using both quality and efficiency metrics:

- **Evaluation loss**
- **Perplexity**
- **Tokens/second**
- **Total runtime**
- **Peak GPU memory usage**

These metrics provide a balanced view of the trade-offs between model performance and computational efficiency.

2.4 Hyperparameters

To ensure comparability across the baseline and sparse variants, the same training configuration was used unless otherwise noted. The most important hyperparameters are summarized in Table 2.

Parameter	**Value**
Maximum sequence length (L)	512
Batch size (per device)	8
Training steps	20000
Local window size (w)	32

Table 2: Hyperparameters used in all experiments.

3 Models

3.1 Baseline: Dense Self-Attention

As a reference system, I use the standard BERT-base masked language model with dense self-attention. In this architecture, every token attends to every other token in the input sequence, which results in a computational complexity of $O(L^2)$ with respect to the sequence length L. While this allows the model to capture global dependencies, it also makes training and inference expensive in terms of memory and runtime, especially for long input sequences.

The model is trained with the standard masked language modeling (MLM) objective: 15% of the input tokens are randomly replaced with the special MASK token, and the model is trained to predict the original tokens. The baseline serves two purposes: first, as a strong performance reference for evaluation loss and perplexity, and second, as a comparison point for efficiency metrics such as throughput and GPU memory usage. All results for the sparse attention variants are reported relative to this dense baseline.

3.2 Variant 1: Local Window Attention

The first sparse variant replaces dense self-attention with Local Window Attention. Instead of allowing every token to attend to all other tokens in the sequence, each token only attends to a fixed-size window of neighboring tokens. Formally, for a given position i and window size w, the attention is restricted to the range $[i-w, \ldots, i+w]$. This reduces the computational complexity from $O(L^2)$ to $O(L \cdot w)$, where L is the sequence length.

3.3 Variant 2: Longform Attention

The second sparse variant, Longformer-style Attention, extends Local Window with sparse global connections: each token attends to its local neighborhood $[i-w, \ldots, i+w]$, the CLS token (position 0), and strided anchors every 512 positions. This maintains $O(L \cdot w)$ complexity while enabling long-range information flow.

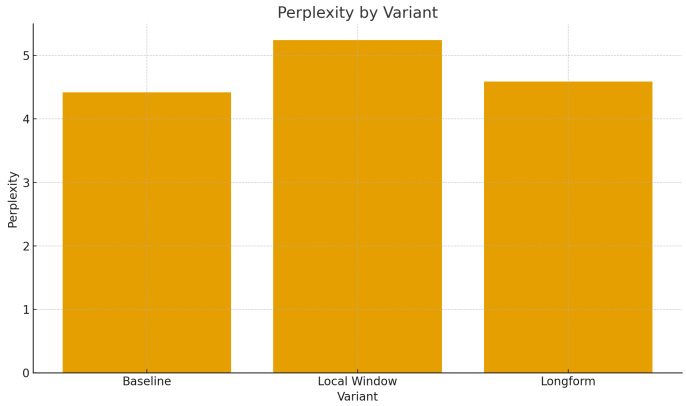
4 Results

4.1 Main Results

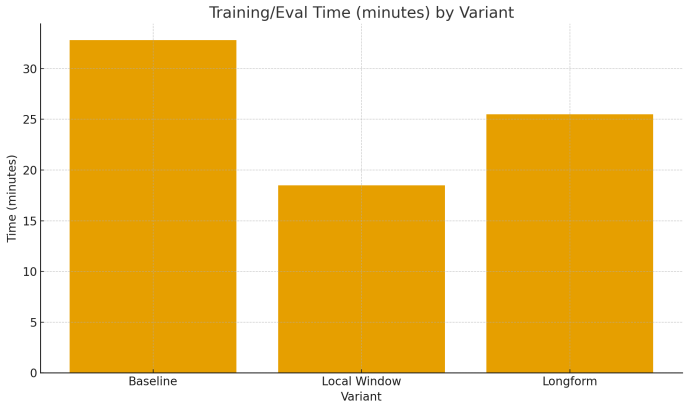
Table 3 summarizes the performance of the baseline and the two sparse attention variants. We report evaluation loss, perplexity, runtime (wall-clock minutes), throughput (tokens per second), and peak GPU memory.

Model	Eval Loss	Perplexity	Runtime (min)	Tokens/sec	Peak GPU Mem (GB)
Baseline (Dense)	1.4868	4.42	32.8	83,318	9.65
Local Window	1.6573	5.24	16.4	110,928	3.18
Longform	1.5228	4.59	25.5	101,839	7.88

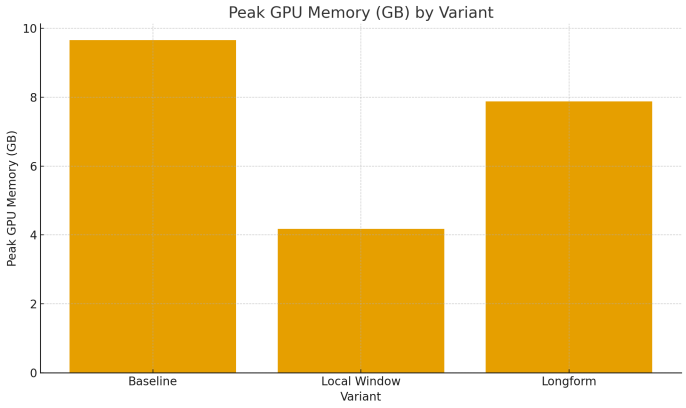
Table 3: Comparison of baseline and sparse attention variants.



(a) Perplexity.



(b) Runtime in minutes.



(c) Peak GPU memory in GB.

Figure 1: Comparison of quality and efficiency across baseline and variants.

5 Analysis

The results highlight clear differences between the dense baseline and the two sparse attention variants in terms of both model quality and efficiency.

5.1 Model Quality

The dense baseline achieves the lowest perplexity (4.42), serving as the reference point. Local Window suffers an 18.9% degradation in perplexity (5.24), which can be attributed to severe context limitation: with a window size of $w = 32$, each token attends to only ≈ 65 tokens out of the 512-token sequence ($\approx$13% coverage). This prevents the model from capturing dependencies beyond immediate neighborhoods, which is critical for language modeling tasks that require discourse-level coherence.

By contrast, Longform shows only a modest 3.8% degradation in perplexity (4.59). Its use of sparse global connections (the `[CLS]` token and strided anchors) effectively approximates full attention for masked language modeling. The `[CLS]` token provides a global information hub, while strided anchors enable multi-hop reasoning across distant positions. This observation aligns with findings by **?**, who reported that structured sparsity preserves quality even with reduced complexity.

5.2 Efficiency

The efficiency gains of sparse attention are asymmetric. Local Window reduces peak GPU memory usage by about 67% (3.18 GB vs. 9.65 GB) and halves runtime (16.4 min vs. 32.8 min). However, these savings come at the cost of substantial quality loss, making it a poor trade-off. Longform, on the other hand, achieves an 18% memory reduction (7.88 GB vs. 9.65 GB) and improves throughput (101k vs. 83k tokens/sec), while maintaining perplexity within 3.8% of the baseline. Runtime reflects this balance, with Longform finishing in 25.5 min—faster than the baseline but slower than Local Window.

5.3 Trade-offs

Taken together, these results illustrate the limits of pure locality and the promise of structured sparsity. Purely local attention is insufficient for language modeling: it yields large efficiency gains but nearly 20% worse perplexity. Longform demonstrates that adding a small number of global connections recovers most of the lost quality while still delivering meaningful efficiency improvements. In practice, this makes Longform a favorable alternative to dense attention: it surpasses the baseline in throughput, reduces memory use, and sacrifices only a marginal amount of quality.

6 Conclusion

These findings highlight that sparse attention can provide meaningful efficiency gains without a severe drop in performance, especially when local context is augmented with sparse global connections. At the same time, the experiments were limited to two sparse variants and a single dataset subset. Future work should extend this analysis to additional attention patterns (e.g., random sparse, BigBird), larger corpora, and downstream tasks to evaluate the generality of these trade-offs.

7 References

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).